

Email Spam Detection

End-to-End Spam Classification Project — Build Guide

PROITBRIDGE · AI/ML COURSE · NLP PROJECT 2

About This Project

This guide walks through building a complete email spam detection system from scratch in Python. Starting from a labeled dataset of short emails, you set up a clean project structure, load the data, clean each email's text using NLP techniques (lower-casing, punctuation removal, stop-word removal and lemmatization with spaCy), convert the text into numerical features with CountVectorizer (the Bag-of-Words model), then train and evaluate a Multinomial Naive Bayes classifier that labels every email as HAM (legitimate) or SPAM (junk/phishing).

By the end you will have a trained model, a command-line prediction script, and a FastAPI web service exposing a /predict endpoint — the hand-off a monitoring dashboard or webapp will later call — plus a quarantine step that moves suspicious emails out of the inbox and holds them for review. You build and train the model locally from this guide first; the webapp and monitoring dashboard are built on top of it afterwards. The project is organized into the eleven stages of the machine-learning lifecycle, each building on the last.

What you'll build: *a project scaffold, a data-loading and preprocessing pipeline, a CountVectorizer Bag-of-Words feature pipeline (with an optional TF-IDF alternative), a trained Naive Bayes spam classifier, an evaluation report, an interactive prediction tool, a deployable REST API, and a quarantine step that feeds the monitoring dashboard you build later.*

Topics Coverage Map

Every concept from the NLP Foundations, Preprocessing & Vectorization syllabus and the stage of this guide where you put it into practice. CountVectorizer is the primary vectorizer for this project (per the project architecture); TF-IDF is covered as the weighted alternative.

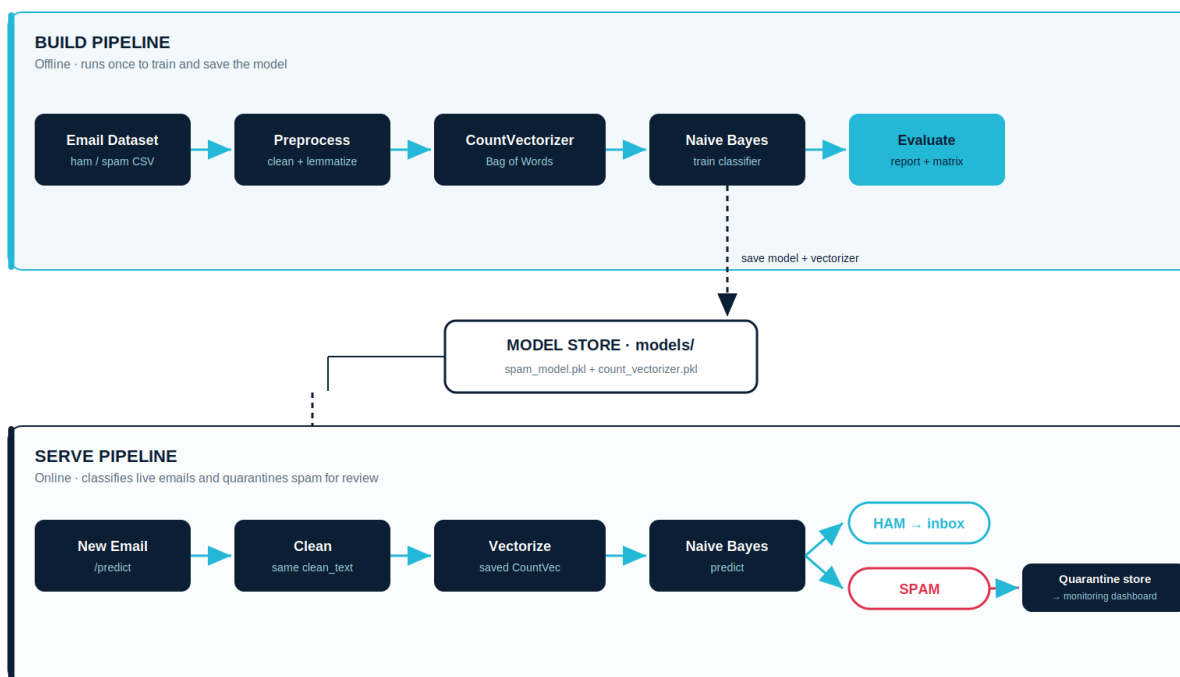
Topic	Where	How it's covered
Introduction to NLP	About, Stage 1	Overview of turning raw, messy email text into machine-ready numerical features.
NLP Pipeline	Stage 4	process_dataset() runs the full clean-then-save pipeline over the dataset in one pass.
Text Cleaning & Lower Casing	Stage 4	str(text).lower() plus regex cleaning inside clean_text() normalize the text.
Punctuation & Stop Words	Stage 4	re.sub(r"^[^a-zA-Z]", ...) strips punctuation; token.is_stop removes common stop-words.
Tokenization	Stage 4	nlp(text) runs spaCy, which splits each email into individual tokens.

Topic	Where	How it's covered
Stemming & Lemmatization	Stage 4	<code>token.lemma_</code> reduces each word to its base form (lemmatization, the more precise of the two).
Preprocessing Pipeline	Stage 4	<code>clean_text</code> and <code>clean_batch</code> applied to the whole dataset in one pass.
Corpus & Vocabulary	Stage 4	<code>CountVectorizer</code> learns the vocabulary from the cleaned email corpus.
N-Grams	Stage 4	<code>ngram_range=(1,2)</code> captures both single words and two-word phrases as features.
Bag of Words	Stage 4	<code>CountVectorizer</code> IS the Bag-of-Words model — each email becomes a vector of word counts.
Count Vectorizer	Stage 4	The core vectorizer for this project; turns cleaned text into integer word-count vectors.
TF · IDF · TF-IDF	Concept (Stage 4)	The weighted cousin of <code>CountVectorizer</code> ; covered as an alternative that down-weights common words.
Naive Bayes	Stage 5	<code>MultinomialNB</code> — the probabilistic classifier that pairs naturally with word-count features.

System Architecture

SYSTEM DESIGN · HOW THE PIECES FIT

Before building anything, here is the shape of the whole system. It is two pipelines that meet at one shared model store. The build pipeline runs offline and turns the labeled email dataset into a trained model saved on disk. The serve pipeline runs online: it loads that saved model to classify live emails, routes HAM to the inbox and quarantines SPAM for review, and records every caught email so a monitoring dashboard can surface it.



End-to-end architecture — the offline build pipeline (top) trains and saves the model; the online serve pipeline (bottom) loads it to classify incoming emails and quarantine spam for the monitoring dashboard.

Two pipelines, one model store

The build pipeline is the data scientist's work (Stages 1–9). Emails are loaded with their ham/spam labels and cleaned with spaCy; the cleaned text is vectorized with CountVectorizer (or TF-IDF); a Multinomial Naive Bayes model is trained, evaluated, and — when the data is imbalanced — optimized. The trained model and the fitted vectorizer are then saved into the models/ folder — the single source of truth the rest of the system depends on.

The serve pipeline is the online path (Stage 10 onward). A new email arrives at the FastAPI /predict endpoint, is cleaned by the exact same clean_text used in training, transformed by the saved vectorizer, and scored by the saved model. The label is returned as JSON — HAM goes to the inbox, and SPAM is sent to a quarantine store by quarantine_email (Stage 10), where the monitoring dashboard — the webapp built on top of this model — lets an admin review and release it.

Note: Both pipelines must clean and vectorize text the same way — a model only makes sense on features built exactly as it was trained. The model store is what guarantees that: the serve pipeline never re-derives anything, it loads the very vectorizer and model the build pipeline saved.

How the code is organised

Each box above maps to a folder in the project, so the architecture and the repository line up one to one:

- **src/data/** — loading and spaCy cleaning (Stages 3–4).
- **src/features/** — the CountVectorizer and optional TF-IDF vectorizers, built in Data Preparation (Stage 4).
- **src/models/** — building, training, evaluation, and the interactive predictor (Stages 5–8).
- **src/api/** — the FastAPI service that exposes /predict (Stage 9).
- **src/quarantine/** — the quarantine store the monitoring dashboard reads (Stage 10).
- **models/** — the saved artifacts both pipelines share.
- **data/** — the raw source data and the processed, cleaned dataset.

The data flow, one line each:

```
Training:  email_spam_dataset.csv -> clean -> vectorize -> train -> models/
```

```
Serving:   email -> /predict -> clean -> vectorize -> model -> HAM / SPAM  
           -> (if SPAM) -> quarantine store -> monitoring dashboard
```

Stage 1: Business Understanding

Every project starts here: before a line of code, get clear on the problem and on how success will actually be delivered. Organizations receive hundreds or thousands of emails daily, making it difficult for employees to identify spam, phishing, and malicious emails manually. This leads to productivity loss, security risks, and inbox clutter.

The goal: automatically classify incoming emails as Spam or Ham using NLP and Machine Learning techniques, quarantine suspicious emails, and provide a monitoring dashboard — improving email security and operational efficiency across the organization.

Why not just do it by hand or with fixed rules?

The slide for this project lays out exactly why a manual or rule-list approach falls short, and why an NLP model is the right tool. The two columns below are the heart of the business case.

Without NLP (the old way)	With NLP (this project)
Staff manually flag junk mail	Every email auto-classified HAM vs SPAM
Fixed rule-lists miss new spam tricks	Learns new spam patterns from data
Inboxes clog up with junk	Junk filtered to the spam folder automatically
Phishing slips through to users	Risky phishing caught early

Manual flagging does not scale — nobody can read every email — and hand-written rule-lists (“block anything with the word FREE”) are brittle: spammers simply change their wording, and the rules quietly go stale. An NLP model instead learns the statistical fingerprint of spam from real examples, so it generalizes to wording it has never seen and can be retrained as spam evolves.

How the finished system works, end to end:

- A new email arrives — typed into the prediction tool, or sent to the /predict API.
- The text is cleaned exactly the way the model was trained to expect (Stage 4).
- The model classifies it as HAM or SPAM (Stages 5–6).
- HAM is delivered to the inbox; SPAM is quarantined — held out of the inbox for review rather than deleted (Stage 9).
- A monitoring dashboard (the webapp built later) surfaces what was quarantined, so an admin can review trends and release any false positives.

This framing drives later choices — how we treat the two kinds of mistakes in Stage 8 (a real email wrongly quarantined vs. a phishing email that slips through) how we handle imbalanced data in Stage 9 (Model Optimization), and the quarantine hand-off in Stage 10. The scope of this guide is to build and train the model locally; the monitoring dashboard and webapp are built on top of the /predict API afterwards.

Stage 2: Data Collection

We gather everything the project needs before touching the data: a Python environment with the right libraries, a clean folder structure, and the dataset itself.

Set Up the Project

SETUP · ENVIRONMENT

Create the project folder.

Why: A dedicated folder keeps every file for this project in one isolated, organized place.

```
mkdir email_spam_detection
cd email_spam_detection
```

Create virtual environment.

Mac/Linux

Why: A virtual environment isolates this project's libraries so they don't clash with other Python projects on your machine.

```
python3 -m venv .venv
```

Activate:

Why: Activating makes every install and run command use this project's isolated Python instead of the system one.

```
source .venv/bin/activate
```

Windows

Why: Same isolation step for Windows — create the environment, then activate it.

```
python -m venv .venv
.venv\Scripts\activate
```

Install Libraries

Create:

Why: Listing dependencies in one file lets anyone recreate the exact same environment with a single command.

```
requirements.txt
```

Paste:

Why: These are the libraries the project relies on — data handling (pandas, numpy), machine learning (scikit-learn), NLP (nltk, spacy), the web API (fastapi, uvicorn), and plotting (matplotlib, seaborn).

```
pandas
numpy
scikit-learn
nltk
spacy
```

```
regex
joblib
fastapi
uvicorn
matplotlib
seaborn
requests
```

Install:

Why: This installs everything listed in `requirements.txt` in one go.

```
pip install -r requirements.txt
```

Install SpaCy model:

Why: `spaCy` needs its language model downloaded separately; this English model powers the tokenizing and lemmatization used later.

```
python -m spacy download en_core_web_sm
```

Create the Folder Structure

SETUP · PROJECT STRUCTURE

Inside project:

Why: A clear layout separates data, models, notebooks, and source code so the project stays maintainable as it grows.

```
email_spam_detection/
```

```
├── data/
│   ├── raw/
│   ├── processed/
│   └── sample/
├── models/
├── notebooks/
├── src/
│   ├── api/
│   ├── data/
│   ├── features/
│   ├── models/
│   ├── quarantine/
│   └── utils/
├── requirements.txt
├── README.md
└── main.py
```

Create them manually or:

Why: These commands build the whole structure automatically instead of creating each folder by hand.

```
mkdir -p data/raw
mkdir -p data/processed
mkdir -p data/sample
```

```
mkdir -p src/api
mkdir -p src/data
mkdir -p src/features
mkdir -p src/models
mkdir -p src/quarantine
mkdir -p src/utils

mkdir models
mkdir notebooks
```

Add the Dataset

Put the dataset here:

***Why:** Keeping the untouched original in a dedicated raw folder means we always have the source data to fall back to.*

```
data/raw/email_spam_dataset.csv
```

The file has two columns and one row per email:

```
email_text,label
Here is the project update you asked for.,ham
Limited offer!!! Buy now and get 50% discount.,spam
Win cash prizes instantly by replying to this email.,spam
Urgent! Your account has been suspended. Verify immediately.,spam
"Hi, please find the meeting agenda attached.",ham
```

Note: This dataset is already labeled — each email comes with a ham or spam tag. That is the key difference from a sentiment project where labels must be derived from the text: here we skip any labeling step and go straight to cleaning. The label column is the target the model learns to predict.

Stage 3: Data Understanding

Create file:

Why: We keep the data-loading logic in its own module so it can be reused and tested independently.

```
src/data/load_data.py
```

Paste:

Why: This small function reads the CSV into a pandas DataFrame — the starting point for everything that follows.

```
# Lifecycle stage 3 – Data Understanding

import pandas as pd

def load_data(path):
    return pd.read_csv(path)
```

Test It

Create:

Why: main.py is the entry point we use to run and verify each piece as we build it.

```
main.py
```

Why: Loading the data and printing the first rows plus the label counts confirms the file path, the column names, and the ham/spam balance before moving on.

```
from src.data.load_data import load_data

df = load_data("data/raw/email_spam_dataset.csv")

print(df.head())
print(df.columns)
print(df["label"].value_counts())
```

Run:

```
python main.py
```

Expected:

```
email_text  label
...

Index(['email_text', 'label'], dtype='object')

spam      169
ham       151
```

You should see the first 5 rows, the two column names, and a roughly even split between spam and ham.

Note: This dataset is small and nearly balanced (about 169 spam to 151 ham). That makes it ideal for learning the full pipeline quickly. On a real production dataset, ham usually far outnumbers spam — Stage 4 explains how to handle that imbalance when it appears.

Stage 4: Data Preparation

We turn raw emails into model-ready inputs. Because the dataset already carries a ham/spam label, there is no labeling step — the work here is cleaning the email text, applying that across the whole dataset, and finally extracting numerical features (vectorization). All three are data preparation; the model itself comes later.

Clean the Email Text

Create:

***Why:** Raw email text is messy — mixed case, punctuation, exclamation marks, URLs, and filler words — so we clean it to help the model focus on the words that actually separate spam from ham.*

```
src/data/preprocess.py
```

Paste:

***Why:** This function lowercases the text, strips URLs and non-letters, removes stop-words, and reduces each word to its base form (lemma). For spam detection, words like “free”, “win”, “urgent”, and “verify” are strong signals, and lemmatization makes sure “wins”, “winning”, and “won” all collapse to the same feature.*

```
# Lifecycle stage 4 – Data Preparation (text cleaning)
```

```
import re
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

```
def clean_text(text):
```

```
    text = str(text).lower()
```

```
    text = re.sub(r"http\S+", "", text)
```

```
    text = re.sub(r"[^a-zA-Z ]", "", text)
```

```
    doc = nlp(text)
```

```
    words = []
```

```
    for token in doc:
```

```
        if not token.is_stop:
            words.append(token.lemma_)
```

```
    return " ".join(words)
```

```
def clean_batch(texts):

    cleaned = []

    # nlp.pipe streams the texts through spaCy in batches;
    # disabling parser + ner skips the slow components we don't use

    docs = nlp.pipe(texts,
                     disable=["parser", "ner"],
                     batch_size=200)

    for doc in docs:

        words = [t.lemma_ for t in doc if not t.is_stop]

        cleaned.append(" ".join(words))

    return cleaned
```

Why two functions: *clean_text* cleans one email at a time (used by the prediction script and the API). *clean_batch* does the same work but streams many emails through spaCy in batches and skips the parser and NER components — this is what keeps processing fast as the dataset grows. We use *clean_batch* for the full dataset in Data Preparation.

Note: If you ever change *clean_text* after already building the project, you must re-run the Data Preparation, Model Building, and Model Training stages. The saved vocabulary and model are baked from the old cleaning, so predictions stay wrong until you rebuild them.

Test Cleaning

Update `main.py`

Why: *Running it on a sample spam line confirms the cleaning produces tidy, normalized output before we apply it to the whole dataset.*

```
from src.data.preprocess import clean_text

sample = """Limited offer!!! Buy now and get 50% discount."""

print(clean_text(sample))
```

Run:

```
python main.py
```

Expected:

```
limited offer buy get discount
```

Punctuation, the “50%”, and the stop-word “and” are gone; the spam-signal words remain.

Process the Entire Dataset

Create:

Why: *This ties the previous steps together — cleaning the whole dataset in one pass and saving the result so we only do this work once.*

```
src/data/process_dataset.py
```

Why: Wrapping it in a function with print checkpoints makes the pipeline reusable and easy to follow as it runs. We do the fast lowercasing and regex as vectorized pandas operations, then hand only the tokenizing and lemmatizing to spaCy via `clean_batch`.

```
# Lifecycle stage 4 – Data Preparation (full dataset)

import pandas as pd
from src.data.preprocess import clean_batch

def process_dataset():

    print("Script Started")

    df = pd.read_csv("data/raw/email_spam_dataset.csv")

    print("Dataset Loaded")

    # Lowercasing + regex are fast as vectorized pandas ops,
    # so we do them here and leave only tokenizing/lemmatizing to spaCy.

    texts = df["email_text"].astype(str).str.lower()
    texts = texts.str.replace(r"http\S+", "", regex=True)
    texts = texts.str.replace(r"^[a-zA-Z ]", "", regex=True)

    df["clean_email"] = clean_batch(texts)

    print("Cleaning Completed")

    df.to_csv("data/processed/cleaned_emails.csv", index=False)

    print("File Saved")
```

Update main.py

Why: Calling the function from main.py runs the full cleaning pipeline end to end.

```
from src.data.process_dataset import process_dataset

process_dataset()
```

Run:

```
python main.py
```

Output:

```
Script Started
Dataset Loaded
Cleaning Completed
File Saved
```

Check:

Why: Saving the cleaned data means later stages can load it instantly instead of re-running the cleaning every time.

`data/processed/cleaned_emails.csv` created successfully, with a new `clean_email` column next to the original `email_text` and `label`.

A Note on Class Balance

Why: Before feeding the data to a model, we always check how the two classes are spread. One line tells us:

```
print(df["label"].value_counts())
```

Output (this dataset):

```
spam    169
ham     151
```

This dataset is nearly balanced (169 spam to 151 ham), so no special handling is needed here — the model sees plenty of both classes, and we can build, train, and evaluate it (Stages 5–8) exactly as written.

Note: A real inbox is very different: ham usually outnumbers spam many times over. That imbalance can quietly wreck a model — it may score high accuracy by labelling almost everything ham while missing the spam that matters. We deliberately leave that problem aside until after evaluation and tackle it head-on in Stage 9: Model Optimization, where we simulate an imbalanced dataset and fix it.

Vectorize the Text (Feature Extraction)

The final data-preparation step turns the cleaned text into numbers. This is feature extraction — it produces the numerical features the model will later be built and trained on, so it belongs to Data Preparation, not Model Building. No model is involved yet; we are only shaping the inputs. This project uses CountVectorizer (the Bag-of-Words model), with TF-IDF as the weighted alternative.

Create:

Why: CountVectorizer turns each email into a Bag of Words — a vector that simply counts how many times each vocabulary word appears. Spam emails reuse the same tell-tale words (“free”, “winner”, “urgent”, “click”), and raw counts capture that signal directly.

```
src/features/vectorizer.py
```

Paste:

Why: This fits the vectorizer on the cleaned emails and saves it as a reusable artifact, so the exact same transformation can be loaded at training and prediction time. max_features caps the vocabulary to the most common words; ngram_range=(1,2) also captures two-word phrases like “click here” or “gift card”.

```
# Lifecycle stage 4 – Data Preparation (feature extraction)
```

```
import pandas as pd
import joblib
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
df = pd.read_csv("data/processed/cleaned_emails.csv")
df["clean_email"] = df["clean_email"].fillna("")
```

```
vectorizer = CountVectorizer(max_features=5000, ngram_range=(1, 2))
```

```
X = vectorizer.fit_transform(df["clean_email"])

joblib.dump(vectorizer, "models/count_vectorizer.pkl")

print(X.shape)
```

Run:

```
python src/features/vectorizer.py
```

Output:

```
(320, 1400)
```

Example — 320 emails, each turned into a row of word counts across the learned vocabulary (your exact column count will differ).

Check:

```
models/
count_vectorizer.pkl
```

CountVectorizer vs TF-IDF — which to use

Both turn text into numbers; they differ in how they weight words:

- **CountVectorizer** — plain word counts (Bag of Words). Simple, fast, and a natural fit for Naive Bayes, which is built around counting how often words appear in each class. This is the default for this project.
- **TF-IDF** — the same idea but each count is re-weighted to down-play words that appear in almost every email and boost rarer, more distinctive ones. It can help when very common words drown out the signal.

To try TF-IDF instead, swap the import and class — everything else stays the same:

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
```

Note: Keep one vectorizer live at a time and always pair it with a model trained on it. A model trained on CountVectorizer features will give meaningless predictions on TF-IDF features, and vice-versa — the serve pipeline must load the very vectorizer the model was trained with.

Stage 5: Model Building

With the features prepared in Stage 4, Model Building is where we choose and configure the algorithm that will learn from them. Note the boundary: turning text into numbers was feature extraction (Stage 4); here we only select and set up the model. Nothing is fitted yet — that happens in Stage 6.

For spam classification we build a Multinomial Naive Bayes model.

***Why Naive Bayes for spam:** it works out the probability that an email is spam from how often each word shows up in spam vs ham, then picks the more likely class. It is fast to train, needs little data, handles thousands of word-features without strain, and has been the workhorse of spam filters for decades — which is exactly why the project architecture pairs it with CountVectorizer.*

Define the Model

Create:

***Why:** Keeping the model definition in its own module means the algorithm and its settings live in one place, so training, evaluation, and any later retraining all build the exact same model.*

```
src/models/build_model.py
```

Paste:

***Why:** build_model() returns a fresh, untrained Multinomial Naive Bayes classifier. alpha is Laplace smoothing — it adds a tiny count to every word so a word never seen in one class during training does not force a zero probability. Stage 6 calls this to get the model, then fits it.*

```
# Lifecycle stage 5 – Model Building

from sklearn.naive_bayes import MultinomialNB

def build_model():

    # alpha = Laplace smoothing; keeps unseen words from zeroing a class
    return MultinomialNB(alpha=1.0)
```

Note: Model Building is also where you would swap in or compare other classifiers — for example LogisticRegression(class_weight="balanced") or LinearSVC() — by returning a different model from build_model(). Because the features are prepared separately in Stage 4, only this one function changes; the rest of the pipeline stays the same.

Stage 6: Model Training

Create:

Why: This is where the model defined in Stage 5 is actually fitted to the features prepared in Stage 4, then saved. Training consumes the feature artifacts from earlier stages — it does not create them.

```
src/models/train.py
```

Why: We load the cleaned data and the vectorizer saved in Stage 4 (we transform with it, never re-fit — fitting was a data-preparation step), build the untrained model from Stage 5, split into train and test sets, fit the model, and save it. `stratify=y` keeps the ham/spam ratio the same in both splits.

```
# Lifecycle stage 6 – Model Training

import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from src.models.build_model import build_model

df = pd.read_csv("data/processed/cleaned_emails.csv")
df["clean_email"] = df["clean_email"].fillna("")

# Load the vectorizer prepared in Stage 4 – transform only, never re-fit.
vectorizer = joblib.load("models/count_vectorizer.pkl")

X = vectorizer.transform(df["clean_email"])
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Build the untrained model from Stage 5, then fit it here.
model = build_model()

model.fit(X_train, y_train)

joblib.dump(model, "models/spam_model.pkl")

print("Training Completed")
```

Note: The vectorizer is loaded here, not created — it was fitted and saved during Data Preparation (Stage 4). Training only fits the model. This keeps a clean line between preparing features and building/training the model, and guarantees the serve pipeline later uses the very same vectorizer.

Run Training:

```
python src/models/train.py
```

Output:

Training Completed

Stage 7: Model Testing

Create:

Why: *This gives a quick, interactive way to try the trained model on your own typed emails.*

```
src/models/predict.py
```

Why: *It loads the saved model and vectorizer, then loops so you can keep entering email text and instantly see the predicted label. Each email is passed through `clean_text` first, so the input is processed the exact same way as the training data — without this, the model sees a different kind of text than it learned from and predictions become unreliable.*

```
# Lifecycle stage 7 – Model Testing

import joblib
from src.data.preprocess import clean_text

model = joblib.load("models/spam_model.pkl")

vectorizer = joblib.load("models/count_vectorizer.pkl")

while True:

    email = input("\nEnter Email: ")

    X = vectorizer.transform([clean_text(email)])

    prediction = model.predict(X)

    print("\nPrediction:", prediction[0])
```

Run:

```
python -m src.models.predict
```

Test:

```
Enter Email:
Win a free iPhone now! Click here to claim your prize.

Prediction:
spam
```

Stage 8: Model Evaluation

Create:

Why: Before trusting the model, we measure how accurately it predicts on data it never saw during training.

```
src/models/evaluate.py
```

Why: Loading the saved model and vectorizer and scoring the held-out test set tells us how well it really performs, class by class.

```
# Lifecycle stage 8 – Model Evaluation

import pandas as pd
import joblib

from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/processed/cleaned_emails.csv")
df["clean_email"] = df["clean_email"].fillna("")

model = joblib.load("models/spam_model.pkl")

vectorizer = joblib.load("models/count_vectorizer.pkl")

X = vectorizer.transform(df["clean_email"])
y = df["label"]

_, X_test, _, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

predictions = model.predict(X_test)

print(classification_report(y_test, predictions))

# Fix the label order so the grid is readable and the
# spam class always sits in the top-left corner.

labels = ["spam", "ham"]

cm = confusion_matrix(y_test, predictions, labels=labels)

cm_table = pd.DataFrame(
    cm,
    index=["actual_spam", "actual_ham"],
    columns=["pred_spam", "pred_ham"])

print("\nConfusion Matrix (rows = actual, columns = predicted)")
print(cm_table)
```

***Why:** The classification report gives summary scores, but the confusion matrix shows the raw counts behind them — exactly which emails were classified correctly and which were confused for the other class. We fix the label order to spam, ham so the spam class lands in the top-left corner, where the numbers that matter most are easy to read off.*

Run:

```
python src/models/evaluate.py
```

Expected:

***Why:** Precision, recall, and F1-score show how well the model does for each class, not just overall.*

```
precision
recall
f1-score
```

Reading the Confusion Matrix for Our Business Problem

Spam detection has two very different kinds of mistakes, and they do not cost the same. The confusion matrix lets us see both directly. The example grid below shows the layout:

```
Confusion Matrix (rows = actual, columns = predicted)
               pred_spam  pred_ham
actual_spam      30         3
actual_ham       1        30
```

Two cells deserve close attention:

- **actual_spam → pred_ham (top-right):** spam the model missed and let into the inbox. When that missed email is phishing, this is the dangerous error — a scam reaches the user. These are the false negatives, and catching them is the whole point of the project.
- **actual_ham → pred_spam (bottom-left):** a legitimate email wrongly sent to junk. This is the costly error in the other direction — the user may never see an important message. These are the false positives.

Unlike a project where one error type clearly dominates, spam filtering must balance both: let too much spam through and phishing reaches users; junk too much ham and people lose real mail and stop trusting the filter. The summary metrics map onto this directly:

- **Recall on spam** — of all real spam, how much did we catch? Low recall means phishing slips through.
- **Precision on spam** — of everything we flagged as spam, how much truly was? Low precision means real emails are being junked.

A good spam filter keeps both high, with a slight lean toward protecting ham — most teams would rather see a little spam in the inbox than lose a real email. Because this system quarantines rather than deletes (Stage 10), a wrongly flagged email is only held for review and can be released from the dashboard, which softens the cost of a false positive — though it is still worth keeping low. Watch both numbers and tune the class weighting from Stage 4 to strike the balance your organization needs.

Stage 9: Model Optimization

Everything up to here assumed a balanced dataset — our emails split almost evenly into spam and ham, so the model saw plenty of both. Real data is rarely that kind. A real inbox is overwhelmingly ham, with spam only a small slice. This stage answers the question that imbalance raises: what happens to the model when one class is rare, and how do we fix it?

The danger of imbalance is a sneaky one. If 95% of emails are ham, a lazy model can label everything ham, score 95% accuracy, and still catch zero spam. The headline number looks great while the model is useless at its actual job. Optimization is where we detect that trap and correct it. The technique we teach here is random oversampling.

Install the Tool

Why: imbalanced-learn (imblearn) adds resampling tools that slot straight into the same fit workflow we already use — nothing else about the pipeline changes.

```
pip install imbalanced-learn
```

Simulate an Imbalanced Dataset

Create:

Why: Our real dataset is balanced, so oversampling would do nothing visible on it. To see both the problem and the fix, we deliberately create an imbalance: keep all the ham, but cut the spam down to a handful. This mimics a real inbox where spam is rare.

```
src/models/optimize.py
```

Paste:

Why: We load the cleaned data, keep every ham row but only the first 25 spam rows, and check the new counts. Counter prints how lopsided the classes now are.

```
# Lifecycle stage 9 – Model Optimization (handling imbalance)

import pandas as pd
import joblib
from collections import Counter

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from imblearn.over_sampling import RandomOverSampler

from src.models.build_model import build_model

df = pd.read_csv("data/processed/cleaned_emails.csv")
df["clean_email"] = df["clean_email"].fillna("")

# Keep all ham, but only 25 spam -> a deliberately imbalanced set
ham = df[df["label"] == "ham"]
spam = df[df["label"] == "spam"].head(25)
```

```
imbalanced = pd.concat([ham, spam])

print("Class counts:", Counter(imbalanced["label"]))
```

Output:

```
Class counts: Counter({'ham': 151, 'spam': 25})
```

Train a Baseline on the Imbalanced Data

Add to the same file:

***Why:** First we train the model on the imbalanced data as-is and read its spam recall. This is the “before” picture — it shows the model ignoring the rare class. We load the saved vectorizer (Stage 4) and reuse build_model (Stage 5), so only the data has changed.*

```
vectorizer = joblib.load("models/count_vectorizer.pkl")

X = vectorizer.transform(imbalanced["clean_email"])
y = imbalanced["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

baseline = build_model()
baseline.fit(X_train, y_train)

print("--- Baseline (imbalanced) ---")
print(classification_report(y_test, baseline.predict(X_test)))
```

Output (illustrative):

```
--- Baseline (imbalanced) ---
              precision    recall  f1-score
ham              0.94         1.00         0.97
spam             1.00         0.20         0.33
```

Notice the spam recall: 0.20. The model catches almost no spam, yet its accuracy still looks high because it is right about all the plentiful ham. That gap is exactly what imbalance does.

Fix It with Random Oversampling

Add to the same file:

***Why:** RandomOverSampler duplicates rows of the rare class until both classes are equal, so the model can no longer win by ignoring spam. We apply it to the training split only, then retrain the same model on the balanced result.*

```
print("Before:", Counter(y_train))

ros = RandomOverSampler(random_state=42)
X_res, y_res = ros.fit_resample(X_train, y_train)

print("After: ", Counter(y_res))

optimized = build_model()
optimized.fit(X_res, y_res)
```

```
print("--- After oversampling ---")
print(classification_report(y_test, optimized.predict(X_test)))
```

Output (illustrative):

```
Before: Counter({'ham': 121, 'spam': 20})
After:  Counter({'ham': 121, 'spam': 121})
```

```
--- After oversampling ---
              precision    recall  f1-score
ham           1.00         0.97     0.98
spam          0.83         1.00     0.91
```

Same model, same features — only the training data was rebalanced — and spam recall jumps from 0.20 to 1.00. That before/after is the whole point of optimization.

Note: Resample the training split **ONLY**, never the test set. If you balance the data before `train_test_split`, duplicated rows leak into the test set and your scores will look better than the model really is. Always split first, then resample `X_train` / `y_train`.

Note: Oversampling works by duplicating existing rows, so the model can overfit to those repeats. It is the simplest, safest place to start — which is exactly why it leads naturally into the other techniques below.

The Wider Landscape

Random oversampling is one tool among several. Knowing where each fits helps you pick the right one later:

- **Random oversampling** — duplicates minority rows until balanced. Simple and safe; the one we taught here. Risk: mild overfitting to the repeats.
- **Random undersampling** — drops majority rows instead. Avoids duplication but throws away real data, so it is wasteful when you do not have much.
- **SMOTE** — creates synthetic minority rows by interpolating between neighbours rather than copying. Powerful, but on word-count vectors it produces fractional counts (e.g. 0.6 of a word) — emails no one ever wrote — so for text, prefer oversampling.
- **Class weighting** — tell the model to penalise mistakes on the rare class more, with no change to the data. The one-liner `class_weight="balanced"` works on `LogisticRegression` and `LinearSVC`, but not on `MultinomialNB` — for Naive Bayes use `sample_weight` in `fit()` or set `class_prior` instead.

Note: This is a teaching demo, so it prints reports rather than saving a model. In production you would fold the winning technique into `train.py` — for example, resampling `X_train` before `model.fit` — so every retrain stays balanced. The rest of the pipeline is untouched.

Stage 10: Model Deployment (Hand-off)

Create:

Why: *Wrapping the model in a FastAPI service lets other applications — a mail client, a frontend, an inbox rule — request predictions over HTTP.*

```
src/api/app.py
```

Why: *This defines a /predict endpoint that takes email text, cleans it with clean_text (the same step used in training), runs it through the saved vectorizer and model, and returns the label as JSON. If the label is spam, it also sends the email to the quarantine store so it is held for review instead of reaching the inbox. This endpoint is the hand-off the webapp and dashboard will call later.*

```
# Lifecycle stage 10 – Model Deployment (hand-off)

from fastapi import FastAPI
from pydantic import BaseModel
import joblib
from src.data.preprocess import clean_text
from src.quarantine.store import quarantine_email

app = FastAPI()

model = joblib.load("models/spam_model.pkl")

vectorizer = joblib.load("models/count_vectorizer.pkl")

class Email(BaseModel):
    text: str

@app.post("/predict")
def predict(email: Email):

    X = vectorizer.transform([clean_text(email.text)])

    label = model.predict(X)[0]

    if label == "spam":
        quarantine_email(email.text, label)

    return {"label": label}
```

Run:

Why: *Starting the server makes the /predict endpoint live on your machine.*

```
uvicorn src.api.app:app --reload
```

Open:

Why: FastAPI auto-generates interactive docs, so you can test the endpoint right in the browser without writing any client code.

```
http://127.0.0.1:8000/docs
```

Quarantine Suspicious Emails

Create:

Why: This is the “quarantine suspicious emails” piece from the project goal. Rather than pushing an alert anywhere, the moment an email is classified spam this module moves it into a quarantine store — held out of the inbox, but not deleted — so it can be reviewed and released later from the monitoring dashboard.

```
src/quarantine/store.py
```

Why: It appends each quarantined email to a CSV with a timestamp and the predicted label. A CSV keeps the local build simple; when you build the webapp later, swap this for a database table the dashboard reads from — only the storage call changes, not the pipeline.

```
# Lifecycle stage 10 – Quarantine store

import csv
import os
from datetime import datetime

QUARANTINE_FILE = "data/quarantine/quarantine.csv"

def quarantine_email(email_text, label):

    os.makedirs("data/quarantine", exist_ok=True)

    new_file = not os.path.exists(QUARANTINE_FILE)

    with open(QUARANTINE_FILE, "a", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        if new_file:
            writer.writerow(["timestamp", "label", "email_text"])

        writer.writerow([datetime.now().isoformat(), label, email_text])
```

Note: Quarantine is reversible by design — nothing is deleted. That is what makes the monitoring dashboard useful: an admin can scan what was caught, spot false positives, and release legitimate mail back to the inbox. This local CSV store is the seed the webapp dashboard will later read and visualise.

Stage 11: CI/CD

The lifecycle does not end at deployment. CI/CD (Continuous Integration and Continuous Deployment) is what keeps the deployed model healthy over time, with as little manual work as possible.

For this project, a CI/CD pipeline would run automated tests on every code change (does the API still start? does cleaning still strip URLs and punctuation?), retrain the model when enough new emails have accumulated, re-check spam recall and ham precision before letting a new model replace the old one, and redeploy the API and quarantine service automatically once the new model passes. Tools such as GitHub Actions, Docker, and a model registry are commonly used to wire these steps together.

This connects back to where we started in Stage 1. Spammers constantly invent new wording, so a model trained once slowly goes stale — which is exactly the weakness of the fixed rule-lists we set out to replace. CI/CD turns the lifecycle you just worked through by hand into an automatic loop, so the filter keeps learning new spam patterns from data and keeps quarantining new spam without anyone re-running every stage.